

An Explainable and Cost-Efficient AI Resume Screening Framework Using OCR-Aware Parsing, TF-IDF Similarity, and Bounded Weighted Scoring

Mrs. Gull Kaur* <i>Assistant Professor</i> <i>Dept. of Computer Science & Engg.</i> <i>Delhi Technological University</i> <i>New Delhi, India</i>		
Rohan Garg 2K22/CO/375 <i>Dept. of Computer Science & Engg.</i> <i>Delhi Technological University</i> <i>New Delhi, India</i>	Punit Sukhani 2K22/CO/353 <i>Dept. of Computer Science & Engg.</i> <i>Delhi Technological University</i> <i>New Delhi, India</i>	Saksham 2K22/CO/395 <i>Dept. of Computer Science & Engg.</i> <i>Delhi Technological University</i> <i>New Delhi, India</i>

Abstract— The first stage of recruitment is increasingly mediated by automated resume screening systems. However, many available systems occupy two unsatisfactory extremes: simple keyword filters are fast but brittle, while transformer-based or large language model approaches provide richer semantic matching at higher computational cost and with reduced transparency. This paper presents a lightweight, explainable, and OCR-aware resume screening framework for first-pass candidate ranking. The proposed system accepts resumes in heterogeneous formats, normalizes text through digital parsing and optical character recognition fallback, extracts technical skills, soft skills, and years of experience through a rule-guided natural language processing layer, computes contextual relevance through TF-IDF cosine similarity, and produces a bounded weighted score with plain-language explanations. Five benchmarks evaluate format-bias resilience, keyword-stuffing resistance, mathematical boundary handling, lexical-versus-semantic trade-offs, and batch efficiency. Results show that the proposed Regex+TF-IDF architecture processes 100 resumes in 0.0062 seconds with 0.12 MB peak memory in the measured benchmark, while a Sentence-BERT baseline requires 4.1860 seconds and 7.33 MB. The system is not intended to replace human hiring judgment, but it provides a practical, auditable, and low-cost first-pass screening layer for small and medium-sized organizations.

Keywords— applicant tracking system, cosine similarity, explainable AI, natural language processing, OCR, recruitment automation, resume screening, TF-IDF

I. INTRODUCTION

Digital recruitment has significantly increased the number of applications received for a single job posting. Online job portals allow applicants to apply across locations with very low effort, and organizations often receive hundreds or thousands of resumes for a single opening. Human recruiters therefore spend substantial time performing repetitive first-stage screening. This stage is also vulnerable to inconsistency because reviewers may give excessive weight to formatting, university names, familiar employers, or visually prominent sections rather than the actual match between candidate capabilities and role requirements [3], [10].

Applicant Tracking Systems (ATS) were introduced to reduce this workload, but many traditional systems operate primarily through rigid keyword matching. A job description requiring “Python”, “REST”, and “PostgreSQL” may cause the system to search for these exact terms in candidate resumes. This approach is computationally inexpensive, but it creates false negatives when a candidate uses alternate wording, abbreviations, or semantically related phrasing. It is also vul-

nerable to keyword stuffing, where candidates insert repeated job-description terms into resumes in order to inflate match scores.

Recent recruitment systems increasingly use transformer-based semantic embeddings and large language models. These methods can map related expressions such as “RESTful APIs” and “REST” into nearby semantic spaces, reducing lexical brittleness [6], [12]. However, they introduce two practical challenges. First, they increase latency, memory consumption, and deployment cost, particularly when processing large batches. Second, they often behave as black-box scoring layers. A recruiter may receive a high or low match score without a clear, auditable explanation of which job requirements were satisfied, missing, or over-weighted. In hiring contexts, such opacity is not merely inconvenient; it can reduce trust and make candidate rejection decisions difficult to justify [8], [14].

This paper presents a practical middle-ground architecture for first-pass resume screening. The proposed framework combines deterministic skill extraction, TF-IDF cosine similarity, bounded weighted scoring, optical character recognition fallback, and rule-based explainability. Rather than claim-

ing complete semantic understanding, it is designed around three constraints common in small and medium-sized enterprise recruitment: speed, low deployment cost, and auditability. Each score is converted into a plain-language explanation that lists matched skills, missing skills, detected soft skills, experience fit, and bonus skills.

The main contributions of this work are:

- an end-to-end OCR-aware resume screening pipeline that handles text, PDF, and image resumes using digital extraction followed by OCR fallback;
- a bounded weighted scoring model that combines technical skills, soft skills, experience, contextual similarity, and capped extra-skill bonuses;
- an explainable AI layer that converts score components into recruiter-readable justifications;
- a benchmark-driven comparison of the lightweight Regex+TF-IDF design against semantic embedding alternatives with respect to robustness, efficiency, and interpretability.

II. BACKGROUND AND RELATED WORK

A. Keyword-Based Resume Screening

Early ATS solutions used Boolean search and keyword frequency counts to match resumes against job descriptions. Such systems are simple to implement and can process large document batches quickly [1], [3]. Their weakness is that they treat language as exact strings. A candidate who writes “backend service development” may be relevant to a backend engineering role, but a strict keyword system may not reward that expression unless it contains the exact expected keywords. Rigid matching also encourages applicant behavior that optimizes for the parser rather than for truthful skill representation.

B. Classical NLP and Vector-Space Models

Classical machine learning methods provide a middle layer between exact keyword search and deep semantic modeling. TF-IDF assigns greater importance to terms that are frequent in a document but not common across the corpus. When used with cosine similarity, it provides a fast numerical estimate of textual relevance [7], [10]. In resume screening, TF-IDF is useful because it reduces the influence of generic terms and can penalize verbose, unrelated content. Rule-guided NLP can complement TF-IDF by extracting structured attributes such as skills and years of experience [16].

C. Transformer-Based Semantic Matching

Transformer-based models such as BERT and Sentence-BERT improve semantic matching by embedding sentences into vector spaces where related meanings can be compared [6], [12]. Recent recruitment systems use such embeddings, LLM feedback loops, and multi-agent frameworks to match candidate profiles and job descriptions more flexibly [2], [5], [11], [13]. These models are valuable when candidates and recruiters use different vocabulary for the same competence. However, they demand more memory and computation than lexical approaches, and they may not naturally produce explanations that are easy for non-technical HR users to audit.

D. Explainable AI in Recruitment

Explainable AI is especially important in hiring because ranking decisions can affect career opportunities [8], [14].

General-purpose methods such as SHAP and LIME can help analyze complex models, but their outputs may be too technical for everyday recruitment workflows [9]. A recruiter usually needs concrete statements such as “matched two out of four required skills” or “missing PostgreSQL and REST”, not only feature-attribution plots. The present system therefore uses a rule-based explanation layer aligned with the internal score dictionary.

E. Format Bias and OCR-Based Parsing

Resume screening systems can unintentionally penalize candidates because of file format rather than content. A parser may fail on scanned PDFs, image resumes, or visually designed resumes with non-linear layouts [7], [15]. OCR fallback reduces such format bias by recovering text when native extraction fails. Although OCR can introduce character-level noise, image preprocessing such as grayscale conversion, binarization, and sharpening can improve extraction quality for standard resume images.

III. PROBLEM FORMULATION

Let $R = \{r_1, r_2, \dots, r_n\}$ be a batch of resumes submitted for a job description J . The objective is to produce a ranked list L where each candidate receives a bounded match score $S_i \in [0, 100]$ and an explanation E_i . The screening task is formulated as an auditable first-pass filtering problem, not as a fully autonomous hiring decision:

$$f(r_i, J, W) \rightarrow (S_i, E_i), \quad (1)$$

where W denotes recruiter-configurable scoring weights. Each processed resume is represented as a structured record containing extracted text, detected technical skills, soft skills, inferred years of experience, contextual similarity, component scores, final score, and explanation. The job description is processed in the same way, but its extracted fields act as requirements. The design targets four requirements: format tolerance, bounded scoring, low computational cost, and human-readable interpretability. The framework deliberately avoids demographic inference and restricts ranking features to text-derived role relevance.

IV. PROPOSED SYSTEM AND METHODOLOGY

The proposed architecture is organized into five modules: document parsing, NLP extraction, weighted matching, explainable AI, and visualization. Fig. 1 presents the overall flow from uploaded resumes and job description to ranked output.

A. Document Parsing Layer

The parsing layer receives resumes in text, PDF, Word, or image formats. Native text is read directly. Digital PDF files are processed through PDF text extraction. Word files are parsed through the document XML structure. If extracted text is below a minimum threshold, the document is treated as scanned or parser-resistant and routed to OCR. Image resumes are directly routed to the OCR path.

B. OCR Fallback Layer

The OCR fallback layer applies image preprocessing before character recognition. In the implemented parser, image-based resumes and scanned PDFs are first routed through the available dots.mocr provider. If that provider fails or is disabled, the system falls back to the Tesseract command-line engine. Before Tesseract execution, transparent images are

flattened onto a white background, converted to grayscale, contrast-enhanced, sharpened, and binarized using a fixed intensity threshold. For scanned PDFs, pages are converted to 300 DPI images before OCR. This fallback is intended to prevent candidates from being penalized simply because their resumes are scanned or image-based.

C. NLP Extraction Layer

The normalized text is lowercased and processed by a lightweight NLP module. SpaCy's `en_core_web_sm` model is used for tokenization, lemmatization, stop-word filtering, and named-entity extraction. Technical and soft skills are not extracted through generic NER; they are matched using strict regular-expression word boundaries against curated skill lists. This is done to avoid false positives such as matching a single-letter technical skill inside an unrelated word. The technical list includes terms such as Python, Django, REST, Docker, AWS, PostgreSQL, and related technologies, while the soft-skill list includes terms such as leadership, communication, collaboration, and cross-functional work. Regular expressions identify direct and indirect years-of-experience evidence such as "5 years of experience", "experience: 5 years", and date ranges.

D. Matching and Ranking Layer

The matching layer computes four principal components: technical skill score, soft skill score, years-of-experience score, and contextual similarity. Technical and soft scores are based on coverage of required skills. Experience is capped so that over-qualified candidates do not receive more than the maximum allotted experience weight. Contextual similarity is calculated using TF-IDF vectors and cosine similarity.

E. Explainable AI Layer

The explanation layer receives the internal score dictionary and converts it into plain-language justification. Rather than post-hoc approximating a black-box model, it directly reports the exact symbolic components used in scoring. This produces statements listing matched required skills, missing skills, detected extra skills, experience fit, and score contribution by component.

F. Module Responsibilities

Table I summarizes the responsibility of each module. This decomposition is important because recruitment screening combines document engineering, NLP, mathematical scoring, and user-interface concerns. Keeping these functions separate improves maintainability and allows individual modules to be replaced later.

V. SCORING MODEL AND ALGORITHM

A. End-to-End Screening Algorithm

The complete workflow is shown in Algorithm 1. This algorithm is included explicitly because the project is best understood as a step-wise transformation from raw input files to ranked recruiter-facing output.

B. Parsing and Text Preparation

The parser uses a threshold $\tau = 50$ characters to decide whether native PDF extraction is sufficient. If a PDF returns fewer than τ characters, the document is treated as scanned or parser-resistant and routed to OCR fallback. The parser also checks the PDF magic-byte header so that images renamed

with a `.pdf` extension can be routed to image OCR. After extraction, text is lowercased and whitespace-normalized, while punctuation is intentionally preserved to protect technical tokens such as `C++`, `Node.js`, and experience ranges such as 1-3.

C. Skill and Experience Extraction

Let T_J denote the set of technical skills required by the job description and T_R denote the set extracted from a resume. The technical skill score is:

$$S_{tech} = \frac{|T_J \cap T_R|}{|T_J|} \times 100. \quad (2)$$

Similarly, if P_J is the set of requested soft skills and P_R is the set found in the resume, then:

$$S_{soft} = \frac{|P_J \cap P_R|}{|P_J|} \times 100. \quad (3)$$

When a job description does not explicitly request soft skills, the soft component can be neutralized or assigned according to recruiter settings.

Skill extraction is performed using boundary-aware matching. A direct substring search can create false positives; for example, matching "R" inside "manager" would be incorrect. The extractor therefore treats skills as tokens or multi-word expressions with boundary conditions. Soft skills are also matched explicitly, which keeps the method transparent but limits semantic recall for indirectly stated traits.

Experience is treated as a bounded requirement. If Y_R is detected candidate experience and Y_J is required experience, the experience score is:

$$S_{exp} = \min\left(\frac{Y_R}{Y_J}, 1.0\right) \times 100. \quad (4)$$

The cap prevents a candidate with double the required experience from receiving a mathematically inflated 200% experience score.

D. Contextual Similarity

The system uses TF-IDF vectorization to represent the resume and job description as vectors. The primary implementation uses scikit-learn's `TfidfVectorizer` with English stop-word removal, a maximum vocabulary size of 5000, and unigram-plus-bigram features (1,2). The bi-gram setting captures short technical phrases such as "machine learning" without loading a neural embedding model. Let A be the resume vector and B be the job-description vector. Their contextual similarity is computed by cosine similarity:

$$S_{ctx} = \frac{A \cdot B}{\|A\| \|B\|} \times 100. \quad (5)$$

TF-IDF rewards distinctive terms more strongly than generic words. In combination with cosine normalization, it reduces the benefit of simply making a resume longer or repeatedly inserting a subset of terms, which is useful for resisting simple keyword-stuffing attacks.

E. Bounded Weighted Score

Recruiters may tune component weights before analysis. The default configuration used in the prototype is:

$$W_{tech} = 0.60, \quad W_{soft} = 0.10, \quad W_{exp} = 0.15, \quad W_{ctx} = 0.15, \quad (6)$$

Module	Input and Output	Responsibility
Document parser	File stream $\rightarrow$ raw text	Reads digital documents, detects low-quality extraction, and routes scanned or image-based resumes to OCR fallback.
OCR fallback	Image representation $\rightarrow$ recovered text	Applies preprocessing and optical character recognition to reduce format bias for non-searchable resumes.
NLP extractor	Normalized text $\rightarrow$ structured attributes	Uses SpaCy for linguistic preprocessing and NER, while detecting technical skills, soft skills, and years of experience through curated dictionaries and regular expressions.
Matching engine	Resume attributes and JD attributes $\rightarrow$ score dictionary	Computes technical coverage, soft-skill coverage, experience score, TF-IDF context score, bonus score, and final score.
XAI generator	Score dictionary $\rightarrow$ explanation	Produces recruiter-readable explanations containing matched, missing, and extra skills along with score components.
Dashboard	Ranked JSON records $\rightarrow$ visual interface	Displays configurable weights, ranked candidates, score breakdowns, and explanation panels.

TABLE I: Functional Responsibilities of the Proposed Framework

where:

$$W_{tech} + W_{soft} + W_{exp} + W_{ctx} = 1.0. \quad (7)$$

The final score is:

$$S_{final} = (S_{tech}W_{tech}) + (S_{soft}W_{soft}) + (S_{exp}W_{exp}) + (S_{ctx}W_{ctx}) + B. \quad (8)$$

The extra-skill bonus B is computed from skills that appear in the resume but are not explicitly required by the job:

$$B = \min \left(|T_R - T_J| \times \frac{B_{max}}{10}, B_{max} \right). \quad (9)$$

The cap prevents score inflation when candidates list many adjacent technologies.

F. Explanation and Complexity

The explanation generator reports the match band, final score, experience status, matched skills, missing skills, extra technical skills, and score breakdown. This deterministic template is intentionally direct: a recruiter can immediately see whether a candidate is missing PostgreSQL, REST, or another required capability. Let n be the number of resumes, d the TF-IDF vocabulary size, and k the skill ontology size. Digital parsing is approximately linear in document length, skill matching is proportional to ontology coverage, and final sorting is $O(n \log n)$. OCR is the expensive branch, but it is invoked only for image files or failed digital extraction. This conditional design preserves speed while still reducing format bias.

G. Feature Engineering Rationale

The framework separates hard evidence from soft evidence. Explicit technical skills and years of experience are treated as bounded requirements, while contextual similarity and soft-skill terms help distinguish candidates with similar hard-skill coverage. The extra-skill bonus is separated from the base score so that adjacent skills such as Docker or Redis add value without hiding missing required skills such as PostgreSQL or REST.

VI. IMPLEMENTATION

The prototype was implemented as a modular Python web application. Flask provides the backend server and routing. The document parsing layer uses native file readers and OCR fallback. The NLP layer uses SpaCy tokenization, lemmatization, named-entity extraction, regular expressions, and curated skill

Component	Technology and Purpose
Backend	Python and Flask for routing, upload handling, and session flow.
NLP	SpaCy tokenization, lemmatization, NER, regular expressions, and skill dictionaries for extraction.
Similarity	TF-IDF and cosine similarity for contextual matching.
Parsing	PyPDF2, python-docx, text reading, image parsing, dots.mocr/Tesseract OCR fallback, and PDF-to-image conversion for scanned documents.
Storage	SQLite, SQLAlchemy, Flask-Login, password hashing, and optional Google OAuth SSO for lightweight user/session persistence.
Frontend	Bootstrap, Jinja2, JavaScript, Matplotlib Base64 charts, and Chart.js for dashboards.

TABLE II: Implementation Stack

dictionaries. The matching layer uses scikit-learn TF-IDF vectorization and cosine similarity. The dashboard is built using HTML, CSS, Bootstrap, Jinja2 templates, JavaScript, Matplotlib, and Chart.js.

A. Backend Flow

When a recruiter uploads resumes and a job description, the backend stores uploaded content and weight settings server-side under a UUID key. Each resume is processed through the parser, NLP extractor, matching engine, and explanation layer. Results are returned as structured objects containing candidate name, score components, matched skills, missing skills, bonus skills, detected soft skills, and generated explanation text. The separation of concerns keeps the parser, matcher, and explainer independently testable.

B. Concurrency and User Experience

Batch screening can create long-running requests if all files are processed synchronously. The prototype therefore starts a daemon background thread and lets the analysis page poll a status endpoint while processing continues. The recruiter configures weights, uploads documents, waits for completion, and then reviews ranked results with score explanations. This supports quick triage while preserving transparency.

C. Recruiter-Controlled Weighting

The frontend provides controls for tuning the scoring weights before the run. Four primary sliders share a normalized 100% pool for technical skills, soft skills, experience, and context. A fifth bonus slider is isolated because the bonus is an additive capped offset. This allows technical roles to prioritize hard skills while management-oriented roles can assign more weight to soft skills and experience. Fig. ?? shows the scoring configuration interface.

D. Explainable Output

The result dashboard does not expose only a single final score. It decomposes the score into technical skills, soft skills, experience, context match, and bonus contribution. The expanded explanation panel lists matched and missing job-description skills, which makes the score traceable. Fig. ?? shows the explainable score breakdown interface.

E. Illustrative Processing Case

For a job description requiring Python, Django, PostgreSQL, and REST with three years of experience, a candidate who lists Python, Django, six years of experience, and extra skills such as AWS, Docker, Kubernetes, and Redis receives a technical score of $2/4 = 50\%$. The experience score is capped at 100%, and the extra-skill bonus contributes 2.0% under the configured cap. The final explanation states that the candidate satisfies experience, matches Python and Django, misses PostgreSQL and REST, and has additional adjacent skills. This demonstrates the raw-text to structured-evidence to score-and-explanation flow.

VII. EXPERIMENTAL SETUP

The framework was evaluated using five benchmarks designed around practical recruitment risks: format bias, keyword stuffing, mathematical boundary errors, semantic mismatch, and batch efficiency. The experiments measure technical skill score, soft skill score, experience score, context score, bonus score, final score, execution time, peak memory, and explanation quality. Each benchmark is implemented as a separate test script so that parser behavior, scoring behavior, explainability, semantic trade-offs, and runtime behavior can be reproduced independently.

A. Benchmark 1: Format Bias and OCR Resilience

The same resume content was stored as a text file, digital PDF, and image file. The objective was to test whether different file formats produced substantially different final scores. This benchmark evaluates whether the OCR fallback can recover enough text from an image resume to avoid format-based penalty.

B. Benchmark 2: Dilution and Keyword Stuffing

The job description was configured to require four technical competencies: Django, PostgreSQL, Python, and REST. Three profiles were evaluated: a baseline profile, a verbose profile containing irrelevant hobbies and unrelated text, and a keyword-stuffing profile containing repeated job-description terms. The objective was to test whether contextual TF-IDF prevents irrelevant verbosity and repetition from dominating the final score.

File	Pipeline	Context	Final
TXT	Native digital parsing	66.0%	90.5%
PDF	Digital PDF parsing	65.5%	90.0%
PNG	OCR fallback	56.8%	90.5%

TABLE III: Benchmark 1: Format Bias and OCR Resilience

Profile	Tech	Context	Final
Baseline	100.0%	57.3%	93.6%
Verbose dilution	100.0%	19.0%	87.8%
Keyword stuffing	100.0%	37.4%	90.6%

TABLE IV: Benchmark 2: Dilution and Keyword-Stuffing Resistance

C. Benchmark 3: Explainability and Mathematical Bounds

An edge-case candidate with six years of experience was evaluated against a role requiring three years. The candidate also included extra technical skills not required in the job description. The objective was to verify that experience and bonus components remain bounded and that the system generates clear recruiter-readable explanations.

D. Benchmark 4: Architectural Comparison

The proposed system was compared against a pure Sentence-BERT-style semantic approach and a hypothetical hybrid approach. The test resume used semantically related phrasing such as “RESTful” instead of “REST” and “directed a team” instead of “leadership”. This benchmark highlights the lexical limitations of the lightweight system and the semantic advantage of embeddings.

E. Benchmark 5: Efficiency and Scalability

A batch benchmark of 100 resumes compared the proposed Regex+TF-IDF architecture against semantic embedding alternatives. Execution time and peak memory were measured to evaluate deployment feasibility for cost-sensitive organizations.

VIII. RESULTS

A. Format Bias and OCR Resilience

Table III shows that the image-based resume processed through OCR achieved the same final system score as the native text file. The digital PDF scored slightly lower due to spacing loss during extraction. This result supports the value of OCR fallback for format-bias mitigation.

B. Keyword-Stuffing Resistance

Table IV shows that the technical score remains 100% when all required technical skills are present, but the context score changes depending on relevance and dilution. The verbose profile suffers a context drop because relevant terms are diluted by unrelated content. The keyword-stuffing profile does not receive a perfect context score because TF-IDF weighting and cosine normalization limit the benefit of repeated terms when the overall document distribution remains abnormal.

C. Boundary and Explanation Validation

For the edge-case candidate, the experience score was correctly computed as:

$$\min(6/3, 1.0) \times 100 = 100\%. \quad (10)$$

The candidate did not receive 200% experience credit. Four extra skills produced a 2.0% bonus, remaining below the 5.0% cap. The generated explanation stated the final score, experience match, matched required skills, missing skills, extra

Approach	Tech	Context	Final
Regex+TF-IDF	0.0%	13.4%	17.5%
Sentence-BERT	N/A	58.4%	58.4%
Hybrid future	0.0%	58.4%	24.3%

TABLE V: Benchmark 4: Architectural Comparison

Approach	Time	Peak RAM
Regex+TF-IDF	0.0062 s	0.12 MB
Sentence-BERT	4.1860 s	7.33 MB
Hybrid future	3.7217 s	1.35 MB

TABLE VI: Benchmark 5: Efficiency on 100 Resumes

skills, and score breakdown. This validates the boundary logic and confirms that the system can produce audit-ready reasoning.

This benchmark is important because score inflation is a common problem in naive ranking systems. If every additional year of experience or every additional skill increased the score without limit, the ranking would favor long resumes and over-listed profiles. By capping experience and bonus contributions, the framework rewards qualification without allowing one component to dominate the entire score.

D. Lexical and Semantic Trade-Offs

Table V shows the semantic limitation of the proposed lightweight approach. Because the prototype uses bounded regular expression matching for hard skill requirements, it does not map “RESTful” to “REST” or “directed a team” to “leadership”. The Sentence-BERT-style approach handles this semantic relationship better, but it does not preserve the same hard-boundary explanation structure. The future hybrid approach offers a possible direction by combining strict skill verification with semantic context reward.

E. Efficiency and Scalability

Table VI reports the 100-resume batch benchmark. The proposed approach is substantially faster and lighter than the semantic alternatives. The measured time of 0.0062 seconds and peak memory of 0.12 MB demonstrate why the framework is suitable for first-pass screening in cost-constrained settings.

F. Summary of Empirical Findings

Across the five benchmarks, the framework satisfies its intended design goals. The OCR fallback reduces file-format sensitivity. TF-IDF context scoring discourages irrelevant verbosity and limits the benefit of repeated keywords under the tested adversarial setup. Bounded experience and bonus calculations prevent score inflation. The architecture comparison confirms that the lightweight model sacrifices semantic flexibility, but this weakness is explicitly understood rather than hidden. The efficiency benchmark shows that the lightweight model has a significant deployment advantage for large initial batches.

These findings support the central claim of the paper: a transparent lexical-statistical model can be a strong engineering choice when the system is used as an auditable first-pass filter. The result is not a universal ranking model, but a practical tool that improves speed and consistency while keeping recruiter judgment in the loop.

IX. DISCUSSION

The results indicate that the proposed system is best understood as a first-pass filter, not as a complete hiring intelligence

platform. It can rapidly identify candidates who clearly lack explicit qualifications while preserving an explanation trail. For high-volume job postings, this staged approach is operationally useful: a lightweight lexical system can perform the initial pass quickly, while human review or semantic matching can be reserved for a smaller shortlist.

The explanation mechanism is deliberately simple. Since every score component is computed from an explicit field, the system directly states which requirements were matched and which were missing. This style of explanation is suited to HR users who need actionable justification rather than model-internals visualization. It also helps maintenance because extraction errors can be traced to the skill dictionary, parser output, or normalization rules.

The architecture avoids GPU dependencies and external API calls, making it practical for classroom demonstration, small HR teams, or early-stage organizations. The OCR fallback contributes to fairness by reducing format bias, but fairness in recruitment is broader than file handling [4]. The system should therefore be used to prioritize review, not to reject candidates without human oversight. A future hybrid design could preserve deterministic hard-skill checks while using embeddings for contextual and soft-skill inference.

X. LIMITATIONS

The most important limitation is the lexical gap. The system may fail to reward semantically equivalent phrasing if the exact skill token is absent. For example, “RESTful services” may not match a strict requirement for “REST” depending on ontology design. Similarly, soft-skill evidence may be expressed through narrative rather than direct words such as “leadership”.

The skill ontology is curated and therefore requires maintenance. New technologies, abbreviations, and synonyms must be added to keep the system current. The OCR fallback improves robustness, but OCR can still fail on complex layouts, handwritten content, low-resolution scans, or heavily graphical resumes. The benchmark dataset is also limited and should be expanded with real-world resumes across industries and formats.

Finally, the system measures document-role alignment, not candidate quality in a complete sense. It cannot evaluate interview performance, portfolio depth, communication ability, or cultural fit. Its purpose is to support recruiters in prioritizing review, not to replace human decision-making.

A. Validity Considerations

The benchmark suite is focused and controlled, but it is not a large industrial dataset. Results may differ on highly visual templates, multilingual documents, or specialized industries. The system also depends on ontology coverage; missing aliases such as “Postgres” for “PostgreSQL” can reduce recall. OCR quality depends on image resolution, layout complexity, and scan noise. Finally, timing results should be interpreted as prototype measurements rather than universal constants, although the relative efficiency advantage of TF-IDF over embedding models remains technically expected.

XI. FUTURE WORK

Future work can improve the framework in four directions. First, a controlled synonym expansion layer can reduce lexical brittleness while preserving deterministic explanations. Sec-

ond, a hybrid model can combine rule-based hard-skill verification with Sentence-BERT or another embedding model for the contextual component. Third, the skill ontology can be moved from static dictionaries to an updateable skill graph with aliases and related technologies. Fourth, fairness evaluation can be expanded using larger datasets with format diversity, role diversity, and adversarial resume patterns.

An additional future direction is role-specific calibration. Different job families require different scoring behavior. Engineering roles may emphasize technical skills, while management roles may require more nuanced weighting for communication, leadership, and years of experience. The current weight sliders provide an initial mechanism, but future systems can learn recommended weight profiles from recruiter feedback.

XII. CONCLUSION

This paper presented an explainable and cost-efficient AI resume screening framework for first-pass recruitment filtering. The system combines OCR-aware parsing, rule-guided skill extraction, TF-IDF cosine similarity, bounded weighted scoring, and plain-language explanations. Experimental benchmarks show that the framework handles multiple file formats, resists simple keyword-stuffing behavior, preserves mathematical score boundaries, and processes resume batches with very low time and memory requirements. The approach does not eliminate the semantic advantages of transformer models, but it demonstrates that lightweight methods remain valuable when transparency, cost, and deployment simplicity are primary constraints. The proposed framework is therefore suitable as an auditable screening layer that narrows applicant pools before human review or deeper semantic analysis.

REFERENCES

- [1] "AI-Powered Resume Parsing using Django: A Smart Recruitment Solution," *IEEE Xplore*, 2026.
- [2] "Resume2Vec: Transforming Applicant Tracking Systems with Intelligent Resume Embeddings for Precise Candidate Matching," *Electronics*, vol. 14, no. 4, 2025.
- [3] "Intelligent Resume Analysis and Skill Matching Using NLP," *IEEE Xplore*, 2026.
- [4] "Adversarial Debiasing in Resume Screening for ICT Positions," e-Repository UPF, 2024.
- [5] "AI Hiring with LLMs: A Context-Aware and Explainable Multi-Agent Framework for Resume Screening," *IEEE Computer Society*, 2025.
- [6] "Applying BERT-Based NLP for Automated Resume Screening and Candidate Ranking," 2025.
- [7] "Efficient CV Parsing: Lightweight Models and Heuristic Methods for Diverse and Non-Standard Formats," *IEEE Xplore*, 2025.
- [8] "Explainable artificial intelligence in the talent recruitment process: A literature review," *Cogent Business & Management*, 2025.
- [9] "Mitigating Bias in AI Model Using eXplainable AI in Terms of Hiring Process in the Industry," *IEEE Xplore*, 2025.
- [10] "Benchmarking AI-Driven Resume Screening: An Evaluation of Precision and Efficiency," *IEEE Xplore*, 2025.
- [11] "AI-Driven Resume Analysis and Enhancement Using Semantic Modeling and Large Language Feedback Loops," *ACL Anthology*, 2025.
- [12] "Enhanced Resume Screening for Smart Hiring Using Sentence-Bidirectional Encoder Representations from Transformers," *International Journal of Advanced Computer Science and Applications*, 2024.
- [13] "AI Hiring with LLMs: A Context-Aware and Explainable Multi-Agent Framework for Resume Screening," arXiv:2504.02870, 2025.
- [14] "Enhancing Recruitment Transparency and Efficiency with Explainable AI," *IEEE Computer Society*, 2024.
- [15] "Layout Analysis for Robust Resume Parsing," *IEEE Xplore*, 2024.
- [16] "Resume Parsing and Skill Extraction using Custom Pattern Matching Algorithm and Gemini API," *IEEE Xplore*, 2024.

AI-Powered Resume Screening & Matching Architecture

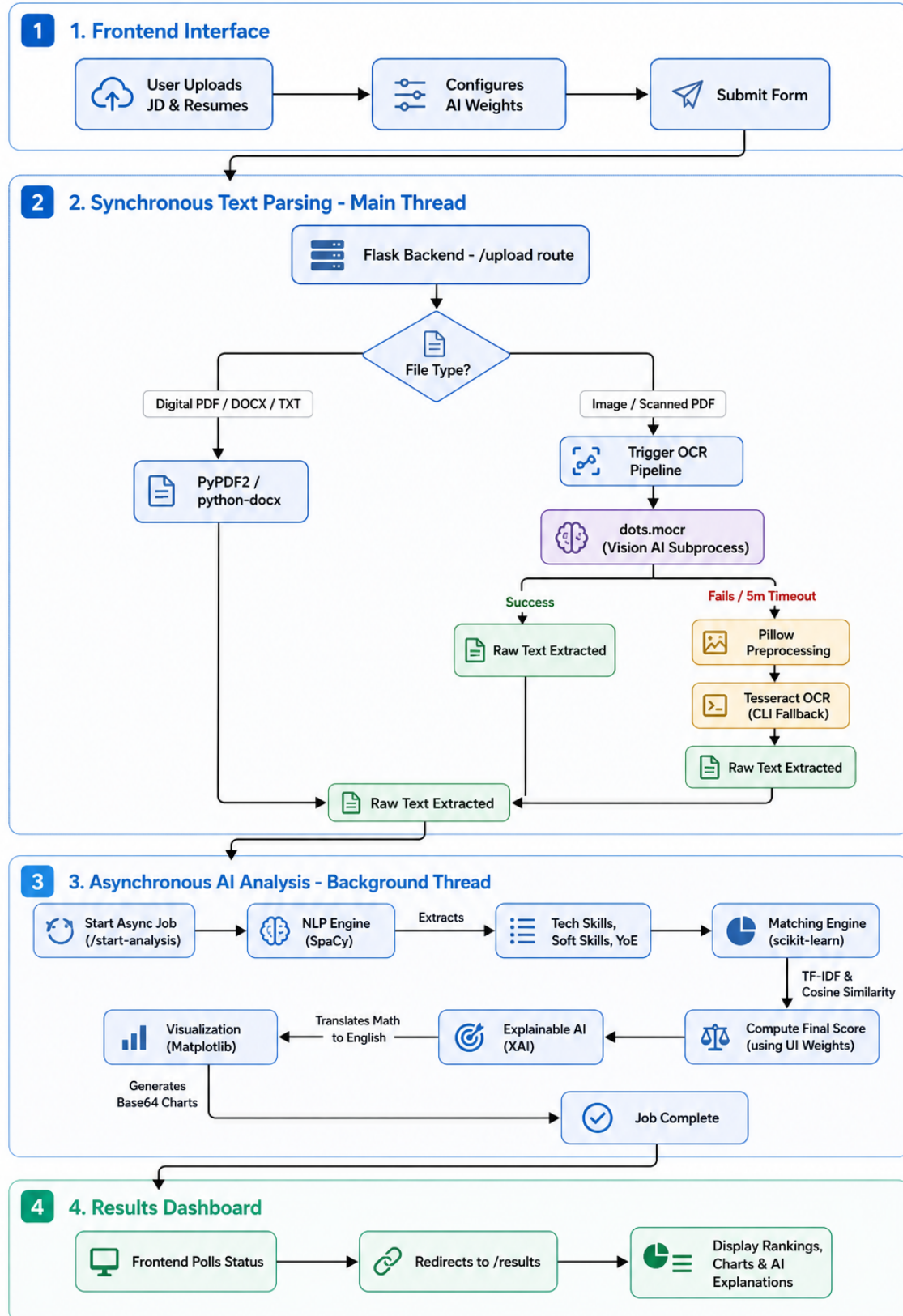


Fig. 1. End-to-end architecture of the proposed AI-powered resume screening system.

Algorithm 1: OCR-Aware Explainable Resume Screening Pipeline

1. **Input:** Resume batch R , job description J , scoring weights $W = (W_t, W_s, W_e, W_c)$, bonus cap B_{max} .
 2. Parse J and extract required technical skills, soft skills, and required experience.
 3. For each resume $r_i \in R$:
 - (a) Detect file type of r_i .
 - (b) If the file is native text or a digital document, perform digital text extraction.
 - (c) If extracted text length is below threshold τ , convert the file to image representation and invoke OCR fallback.
 - (d) If the file is an image, apply OCR using the available OCR provider; if Tesseract fallback is required, flatten transparency, convert to grayscale, boost contrast, sharpen, binarize, and run OCR.
 - (e) Normalize extracted text by lowercasing, removing noisy tokens, and standardizing spacing.
 - (f) Extract candidate technical skills, soft skills, and years of experience using skill ontology and regular expressions.
 - (g) Compute technical coverage, soft-skill coverage, capped experience match, and TF-IDF cosine similarity.
 - (h) Compute capped extra-skill bonus from skills present in resume but absent in the job description.
 - (i) Calculate final weighted score S_i .
 - (j) Generate explanation E_i from matched, missing, bonus, and score-breakdown fields.
 4. Sort all candidate outputs by S_i in descending order.
 5. **Output:** Ranked candidate list $L = \{(r_i, S_i, E_i)\}$ and dashboard visualizations.
-

